Stock Price Prediction

Dev Luhar

University of Warwick

Introduction

Stock price prediction forecasts market behaviour from historical data, though perfect accuracy is impossible.

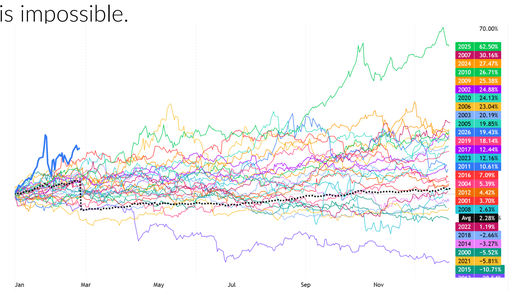


Figure 1. Yearly Gold Prices 2000-2026 [6]

Machine learning can uncover patterns across diverse data, with modelling choices guided by data structure and objectives. This study leans towards time-series stock price prediction, but the framework applies to other prediction tasks as well.

Problem formulation

Before building any prediction models, we need to define a clear prediction target, identify the inputs used and establish a metric to evaluate prediction accuracy.

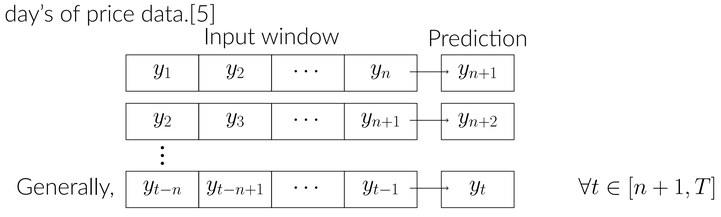
- **Objective:** The aim is to estimate the future price y_t using historical information available up to time $t-1$. The model performance is assessed by comparing predicted prices $\hat{y}_t$ with true prices y_t, using metrics such as root mean squared error and directional accuracy.
- **Time-Series Structure:** Stock prices form an ordered sequence $\{y_t\}_{t=1}^{T}$ with a dependence on time, implying that future prices depend on past observations and underlying market dynamics.
- **Prediction Target:** The model learns a mapping $\hat{f}$, such that the predicted price is given by
$$\hat{y}_t = \hat{f}(y_{t-1}, \ldots, y_{t-n}),$$
where n is the look-back window.
- **Loss Function:** The model's training focuses on minimising prediction error, typically defined as a function of the residual $y_t - \hat{y}_t$, for that we define a loss function to measure it.

Data Collection and Preprocessing

Market data is obtained from reliable financial data providers, in practical implementations, this is achieved using libraries such as `yfinance`.

- The dataset is split into training and test sets in chronological order to preserve temporal structure and prevent information leakage.

- Missing observations, anomalies and non-trading days are processed to ensure a continuous and consistent time series while training.
- A time-series model is trained using overlapping input windows of n consecutive days, to predict the next day's price based on the previous n day's of price data.[5]



Feature Engineering

Feature engineering mitigates gradient instability in sequential models, where gradients can decay to zero (vanishing gradients), preventing learning of long-term dependencies, or grow uncontrollably (exploding gradients), leading to unstable training. To balance expressiveness with stability across different model architectures, we adopt a compact set of informative features.

- **Log returns:** Defined as $r_t = \log\left(\frac{y_t}{y_{t-1}}\right)$, this stabilise the time series and capture proportional price changes.
- **Moving averages:** Computed as $MA_t = \frac{1}{n}\sum_{i=0}^{n-1} y_{t-i}$, this smooths short-term noise and highlight underlying trends.

These features are combined to form the input vector $Y_t = [y_t, r_t, MA_t]$ and the forecasting problem becomes learning the mapping:
$$\hat{y}_t = \hat{f}(Y_{t-1}, \ldots, Y_{t-n}),$$
where future prices are predicted from engineered feature sequences rather than raw price values.

Model Architectures

Rather than fixing a single architecture, models are chosen to match the data structure, prediction objective and computational constraints. Time-series forecasting admits several architectures with distinct inductive biases. Two architectures used for time-series modelling are:

- **LSTM** — Captures short and long-term dependencies in moderate-length sequences; gated memory supports stable learning in noisy financial data.[3]
- **Mamba** — State-space sequence model enabling linear-time inference and long-context modelling; effective when large look-back windows make attention computationally costly.[4]

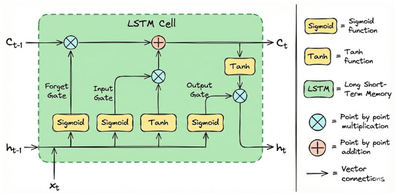


Figure 2. LSTM Architecture [3]

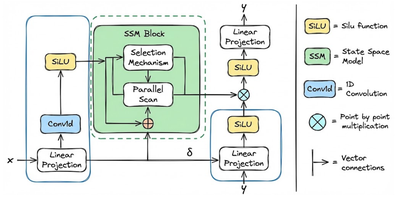


Figure 3. MAMBA Architecture [2]

Training Procedure

Having chosen an LSTM for this study, the training procedure then governs how the model learns from data. Central to this process is the choice of loss function, which defines what constitutes a '*good prediction*'.

- Mean Squared Error (MSE), $\mathcal{L}_{\text{MSE}}$, penalises large deviations in predicted price levels but is sensitive to outliers.
- Quantile Loss, $\mathcal{L}_q$, allows asymmetric penalisation of errors and is useful for modelling downside risk and prediction intervals.
- Huber Loss, $\mathcal{L}_\delta$, provides robustness to outliers while maintaining stable gradients.

In practice, these losses may be combined into a single objective function,
$$\mathcal{L} = w_{\text{MSE}}\mathcal{L}_{\text{MSE}} + w_\delta \mathcal{L}_\delta + w_q \mathcal{L}_q,$$
allowing the optimisation process to reflect the priorities of the chosen prediction objective. Training stability and generalisation are further improved using early stopping, regularisation and adaptive optimisers. [1]

Prediction Analysis

Evaluation Metrics

The assessment of model accuracy and signal quality is objective-dependent. Three standard evaluation metrics are:

- **Root Mean Squared Error (RMSE)**: $\text{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}$ Measures the average magnitude of prediction error in price units.
- **Directional Accuracy (DA)**: $\text{DA} = \frac{1}{T}\sum_{t=1}^{T}\mathbb{I}\left[(y_t - y_{t-1})(\hat{y}_t - y_{t-1}) > 0\right]$ Evaluates how often the model correctly predicts the direction of price movement.
- **Information Coefficient (IC)**: $\text{IC} = \text{corr}(y_t - y_{t-1}, \hat{y}_t - y_{t-1})$ Measures the correlation between predicted and realised returns, assessing signal quality rather than absolute accuracy.

In this study, RMSE and DA were prioritised to evaluate price-level accuracy and directional performance, while IC provides insight into the predictive strength of the model output.

Prediction Analysis

After training, the model is evaluated on unseen time-series data using a rolling-window forecasting strategy. Predictions are generated sequentially and compared against observed prices to assess performance.

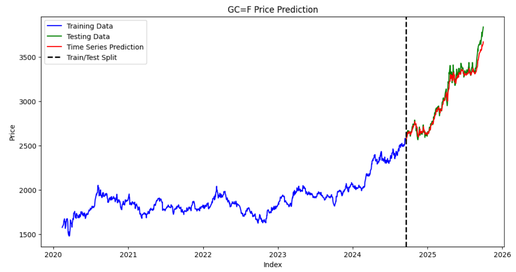


Figure 4. Gold Price LSTM Prediction

Metric	Score
RMSE	55.8
DA	53.9%

Table 1. Gold price LSTM Prediction Scores

Figure 4 compares predicted prices $\hat{y}_t$ with true prices y_t over the evaluation period, highlighting the model's ability to capture broad trends while exhibiting lag during rapid market movements.

Limitations and Conclusions

Financial markets are inherently noisy and non-stationary, meaning relationships learned from historical data may not persist under changing conditions. Models trained solely on price-based features may therefore struggle to adapt to sudden regime shifts.

Performance can be improved through model ensembling, where predictions from multiple architectures are combined using weighted averages to reduce variance and improve robustness. In addition, alternative data sources such as financial news can be incorporated by extracting sentiment signals and integrating them as auxiliary inputs alongside price-based features.

Overall, this framework treats stock price prediction as a sequence of modelling decisions rather than a fixed pipeline, allowing models to be tailored to specific objectives.

References

[1] Remco Duits. Mathematics of neural networks, 2025.

[2] GeeksforGeeks. What is a mamba model?, 2025.

[3] GeeksforGeeks. What is lstm - long short term memory?, 2025.

[4] Albert Gu and Tri Dao. Mamba: Linear-time sequence modelling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[5] Arslan Kamchybekov. How i built a model that predicts stock prices. *Medium*, 2024.

[6] Inc. TradingView. Gold seasonals chart, 2026.